Lecture 2:
The Richness of Data

Why modern data changes what we can know and do

Prof. Martin Shepperd

Image courtesy of Jonathan Weicher

Lecture Poll

Please use your mobile or computer to go to:

pollev.com/mshepperd

We will need it in a few minutes. Thanks!

Lecture protocol

1. I will start at **1005 prompt**; please be ready
2. Be aware, lectures will be recorded
3. Feel free to ask **questions** as we go along or ...
4. ... ask during in a question gap (2-3 per lecture)
5. Be **considerate** of others (fellow students and me) and don't chat.
6. Thanks!

Agenda

1. The richness of Modern Data
2. Data sources
3. Complex data structures in R
4. Reading and writing files in R
5. Databases and R
6. Very large data sets
7. Week 2 goals

1. Why modern data is transformative

Why is modern data so transformative?

— It’s more than size, it’s variety, speed and connections.

— Data shapes decisions and drives discovery.

— Rich data means multiple dimensions, multiple insights.

Volume 📊 – staggering amounts of data (petabytes, zettabytes)
- e.g., social media creates terabytes of content daily

Variety 🌀 – many forms and formats
- e.g., a site may collect transactions, reviews, images and logs

Velocity ⚡ – data moves fast and in real time
- e.g., financial markets in milliseconds, IoT streaming non-stop

Veracity ✅ – quality and trustworthiness
- e.g., social media is noisy but still useful once cleaned

2. Data sources

Where does data come from?



— **Structured vs Unstructured**
Databases vs Social Media Posts

— **Temporal vsSpatial**
Stock data, Log files vs GIS data, Floor plans

— **Human vs Machine**
Surveys vs Sensor readings

— **Static vs Streaming**
Historical datasets vs Live feeds

— **Public vs Private**
Gov data vs Business data

— **Domain-specific**
Images/Videos (medical scans, CCTV)
Text (papers, blogs, reviews)
Biological/Medical (DNA, patient records)

We're living in a golden age of text data

From tweets to research papers, text is everywhere and we now have the tools to analyse it. [1]



[1] A worked example in R or for a very detailed book "Text Mining with R: A Tidy Approach" by David Robinson and Julia Silge (2017) or be patient and wait until Lectures 7 and 8!

3. Complex data structures in R

3.1 Data types recap
3.2 Data frames recap

3.1 Data types (of variables)

— A variable is a named **container** for data; it can be set, modified, referenced or deleted.

— R determines the data type from what you assign/coerce.

```
> n <- 10L          # The L (for Long) makes an integer type
> is.numeric(n)
[1] TRUE
> is.integer(n)
[1] TRUE
> n <- 10.1         # Coerce from integer to numeric
> is.integer(n)
 [1] FALSE
```

Data types matter

```
> # Make a logical vector x
> # T is short for TRUE, F for FALSE
> x <- c(T, T, T, F, F)
> min(x)
```

Wisdom of crowds: What do you think happens?

— Sometimes we can get unexpected results!

— R can surprise you so be careful with your data types.

Scale[3], Data Type and R

Scale	Example	R Data Type
Nominal	Country, Sex	Factor (restricted), Character (unrestricted)
Binary	Yes/No, T/F	Logical (special case of nominal)
Ordinal	Likert scale	Numeric (treated as ordered), Factor (with levels)
Interval	Temperature (°C)	Numeric
Ratio	Length, Weight	Numeric
Absolute	Counts	Integer (with `L` suffix)

[3] Stevens, S. S. (1946). On the theory of scales of measurement. Science, 103, 677-680.

Repeating groups[4]

— vectors are repeating groups of the same type
— lists can mix types
— more generally ...

Table of more complex data structures in R

Dimensions	Homogeneous type	Heterogeneous types
1	vector	list
2	matrix	data frame
n	array	n.a.

[4] See the Modern Data book Chapter 2.4.

3.2 Data frames

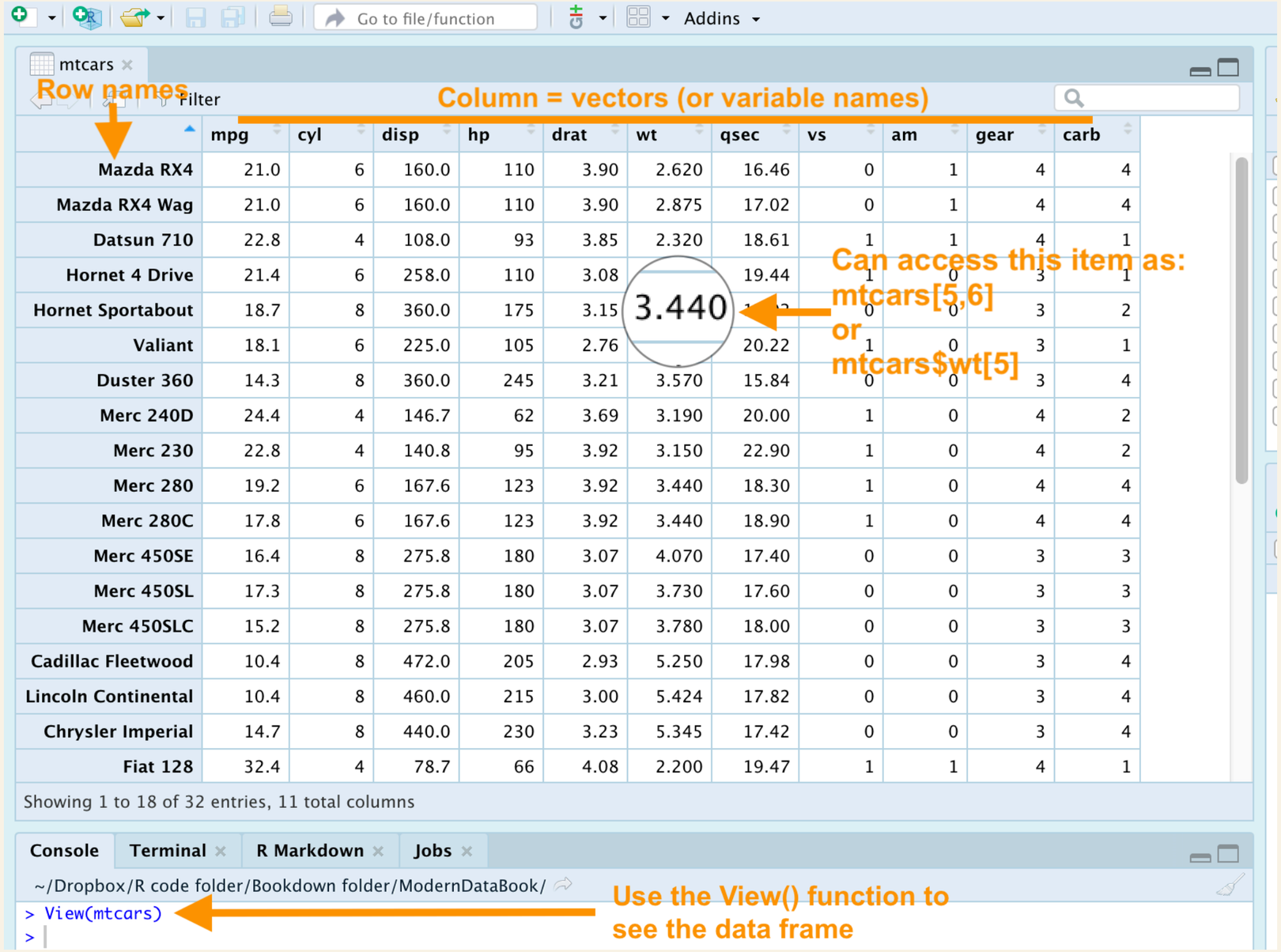
— The data frame is one of the most useful data structures in R

— Analogous to a spreadsheet but you can't directly edit

— Think of a data frame as a collection of aligned vectors (columns)

— Easy to `View()` and manipulate

— Efficient as each column is a vector so simple code and fast processing



	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08		19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440		0	0	3	2
Valiant	18.1	6	225.0	105	2.76		20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1

Data frames and data type

— Very easy to import files into data frames

— Think about the variables (columns) e.g., use `head()` or `str()` to examine them

— Be careful about numeric values/levels for factors (e.g., Gender) **(Why?)**

4. Reading and writing files in R

— Very often, the data we need to analyse is stored in external files.

— Frequently as comma-separated variable (CSV) files

— Many built in functions to help e.g., `read.csv()`

— Great deal of flexibility

R code to import - read.csv()

```
# R code to read a remote file (on GitHub)
# into a data frame pubsDataFrame
# Because the path url is quite long I've
# copied it to a character string fname
# to improve readability

fname <- "https://raw.githubusercontent.com/
mjshepperd/CS5702-Data/master/pubs.csv"
pubsDataFrame <- read.csv(fname,
                          header = TRUE,
                          stringsAsFactors = FALSE,
                          encoding = "UTF-8")
head(pubsDataFrame)
```

The result

```
249  head(pubsDataFrame)
250  ```
```

	pubName <chr>	open <lgl>	town <chr>	weeklySales <int>	foodSales <int>
1	The Dead Albatross	TRUE	Uxbridge	2735	1209
2	The Island Queen	TRUE	Islington	3644	0
3	Johnnys Bar	FALSE	Vladivostok	0	0
4	Red Lion	TRUE	Habrough	3263	*NA*
5	The Crown	FALSE	Haccombe	0	0
6	Royal Oak	FALSE	Haceby	0	0

6 rows

Text encoding standards

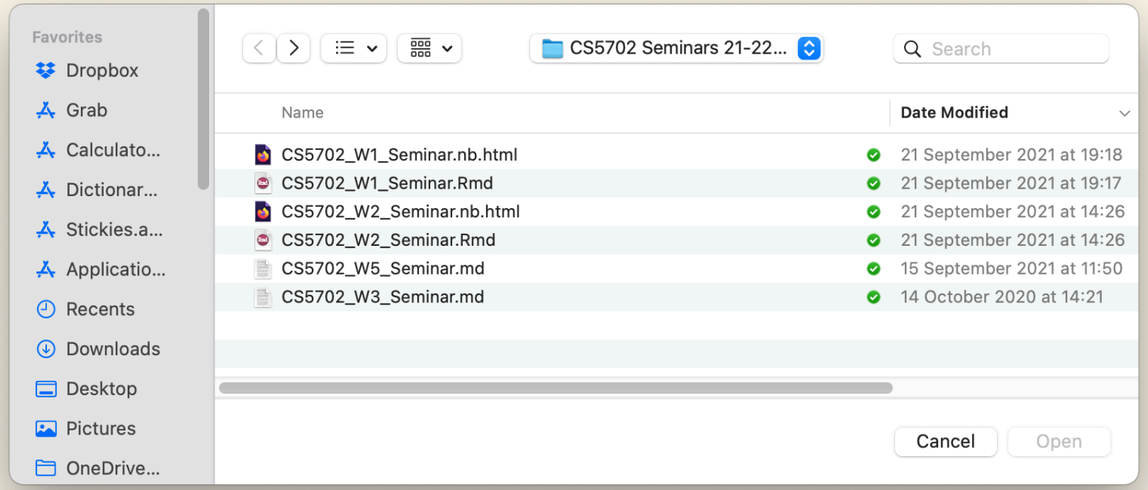
— UTF-8 is a widely used standard that supports all characters, including special symbols and non-English characters.

— It ensures **compatibility** across different platforms and systems.

— **So** use `fileEncoding = "UTF-8"` when importing/exporting CSV files in R to avoid encoding problems.

A useful file reading tip

```
# The user is prompted
# to locate the file
myDF <- read.csv(file.choose())
```



Writing files

Writing files

— Exporting is as important as importing

— Use `write.csv()` to save a data frame

— By default, existing files are **overwritten**

```
write.csv(myDF, "mydata.csv",
                row.names = FALSE),
                append = TRUE)  # override default
```

5. Databases and R

— Data is often stored in **databases** (e.g., SQLite, MySQL, Oracle)

— A database = **collections of tables** (not just one CSV)

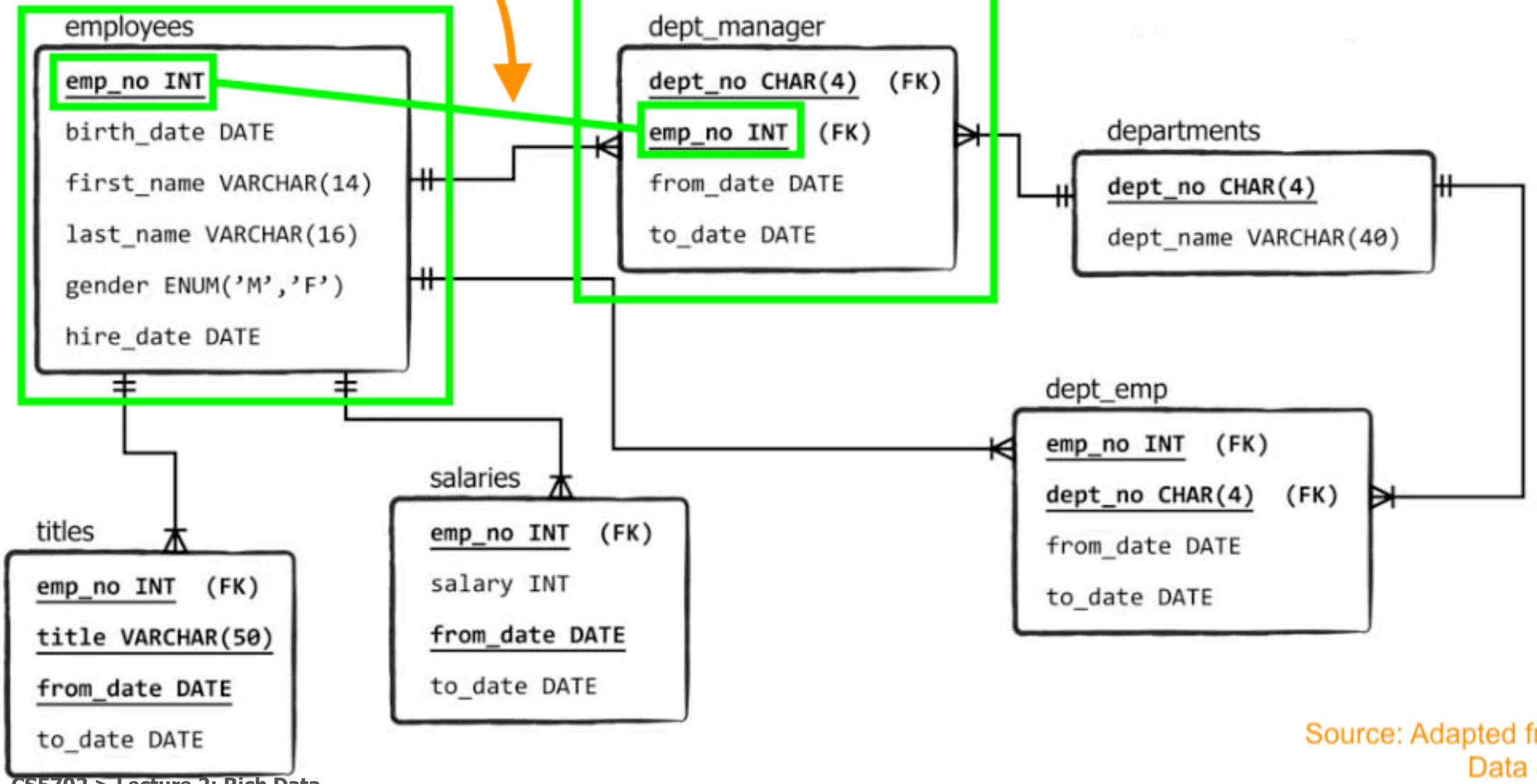
— Goal: organise/join tables into a single rectangular data frame

— Useful for **very large data** → query only what you need

— R connects easily (e.g., `DBI`, `RSQLite`) and lets you embed SQL in code

Database: employees

Joining database tables via Foreign Keys



Source: Adapted from 365 Data Science

6. Very large data sets

— Rise of **"Big Data"** over the last decade

— From terabytes → petabytes → exabytes → now zettabytes

— Global data sphere projected to hit ~180 ZB by 2025

— Growth driven by **IoT, AI, cloud, streaming**

— Challenges: not just volume but also complexity

R and Large Data Set Restrictions

1. R holds data **in memory** (RAM) → typically 8–32 GB on a laptop
2. Rule of thumb: need **~2× RAM** compared to raw data size (manipulations create copies)
3. Import is **slow** for very large files (reading takes longer than analysis)
4. R has a **2 billion element limit** for vectors (≈ 2×10^9)

Big data solutions

1. Use specialised packages (often implemented in C++)
   — e.g., {bigmemory}, {ff}, {data.table} for out-of-memory access
2. Scale up with parallelisation/distribution
   — e.g., cloud compute or Hadoop via {RHadoop}
3. **But** in Modern Data we will focus on **"small data"** (in-memory) where R excels

7. Week 2 goals

By the end of this week you should:
[ ] have an appreciation of the richness of data e.g. list different characteristics and types
[ ] be familiar with some of the major sources of free and publicly available data
[ ] use R to create and manipulate vector and data frame variables
[ ] use R to import and export csv files
[ ] complete the Week 2 worksheet and quiz

Further Reading

1. For more details on R basics, file handling and databases see the Modern Data, chapter two
2. Alternatively see chapters 1, 2 and 4 from Kabacoff (2015) R in Action: Data Analysis and Graphics with R
3. Grolemund and Hadley (2018) R for Data Science, Chapter 13 for more details of working with relational databases such as MySQL and sqlite in R

Questions